PROJECT REPORT

AI CHATBOT FOR FOOD ORDERING

Developed using Python Programming

Project Topic	AI Chatbot for Food Ordering
Programming Language	Python
Project Type	Academic / Internship Project

1. INTRODUCTION

An AI Chatbot for Food Ordering is a computer-based application developed using Python programming and basic chatbot concepts. It helps customers place food orders through a text-based conversation. Instead of manually checking a menu and communicating every detail to restaurant staff, customers can interact with the chatbot to view available food items, select meals, enter quantities and view the total bill.

The chatbot receives a customer's message, checks the input and provides a suitable response. In this project, basic responses are produced using predefined rules and Python conditional statements. The project demonstrates the foundation of a food-ordering chatbot. It can later be enhanced with Natural Language Processing (NLP), a database, online payment facilities and order tracking.

2. OBJECTIVES OF THE PROJECT

• To develop a simple and user-friendly food-ordering chatbot.

• To allow customers to interact with the system through text messages.

• To display different food items and their prices.

• To accept customer food orders and quantities.

• To calculate the total bill automatically.

• To reduce repetitive manual work in the ordering process.

• To provide quick and consistent responses to customer requests.

• To demonstrate the use of Python programming in a practical application.

3. EXISTING SYSTEM

In a traditional food-ordering system, customers usually place orders by speaking to restaurant staff, visiting the restaurant or using a basic ordering form. Staff members note the selected items, check availability, calculate the bill and confirm the order. During busy periods, this process may take more time. Manual entry or calculation can also lead to mistakes.

Disadvantages of the Existing System

• More staff involvement is required for taking orders.

• Customers may have to wait during busy hours.

• Mistakes can occur while recording food items or quantities.

• Manual bill calculation may take additional time.

• The ordering process may become slower when many customers arrive together.

4. PROPOSED SYSTEM

The proposed system is a Python-based food-ordering chatbot. It interacts with customers through text, displays the menu, accepts food selections, checks whether an item is available and calculates the bill. The program provides immediate responses for the supported commands and keeps the ordering steps organized.

The version demonstrated in this report is a basic rule-based prototype. It uses predefined menu data and conditions to respond to customer inputs. It does not independently understand every natural-language sentence or connect to a live restaurant service. These capabilities can be added in a more advanced version.

5. SYSTEM REQUIREMENTS

Software Requirements

Software	Purpose
Python 3	Programming language used to develop the chatbot.
Visual Studio Code / Python IDLE	Used to write, edit and run Python programs.
SQLite (optional extension)	Can store customer and order information.
Windows, Linux or macOS	Operating system for running the program.

Hardware Requirements

• A desktop computer or laptop.

• At least 4 GB RAM is recommended for basic development.

• Keyboard and mouse or touchpad.

• Sufficient storage space for Python and project files.

6. THEORETICAL CONCEPTS

6.1 Artificial Intelligence

Artificial Intelligence (AI) is a field of computer science concerned with creating systems that can perform tasks commonly associated with human intelligence, such as recognizing patterns, making decisions and understanding language. In a food-ordering application, AI techniques can help a system interpret customer requests, recommend food and respond to questions. The programs in this report demonstrate basic chatbot logic; they are not a trained AI model.

6.2 Chatbot

A chatbot is a software application that communicates with a user through text or voice. It receives a message, processes it and returns a response. Some chatbots use fixed rules, while advanced chatbots use machine-learning or language models. The chatbot in this project uses fixed rules to recognize supported words such as 'hello', 'menu', 'pizza' and 'exit'.

6.3 Natural Language Processing

Natural Language Processing (NLP) is a branch of AI that helps computers process human language. NLP techniques can be used to identify important words, determine the intention behind a message and generate an appropriate response. For example, an advanced food chatbot could recognize that 'I would like two pizzas' is an order request. The current prototype uses simple input matching rather than a complete NLP pipeline.

6.4 Python Programming

Python is a high-level programming language known for its readable syntax. It supports variables, data types, dictionaries, conditional statements, loops and functions. These features are used in this project to store menu information, accept customer choices, check availability and calculate order totals.

6.5 Database Management

A database stores information in an organized form so that it can be retrieved and updated. A full food-ordering application may store customer details, menu items, prices and order history. SQLite is one possible database

choice for a small Python application. The main sample programs in this report use in-memory variables and dictionaries; they do not permanently save orders.

7. SYSTEM WORKING

Step 1 – Customer interaction

The customer starts the program and receives a welcome message.

Step 2 – Display menu

The chatbot displays available food items and their prices.

Step 3 – Food selection

The customer enters the name or number of a food item.

Step 4 – Availability checking

The program checks whether the selected item exists in the menu.

Step 5 – Quantity entry

The customer enters the required quantity.

Step 6 – Bill calculation

The program multiplies the item price by the quantity and adds it to the running total.

Step 7 – Order summary

When the customer finishes, the program displays the customer name and total amount.

8. PYTHON PROGRAMS WITH OUTPUT AND EXPLANATION

PROGRAM 1: DISPLAY WELCOME MESSAGE

Aim: To display a welcome message and the main options of the food-ordering chatbot.

Program:

```
print("================================")
print(" WELCOME TO FOOD ORDERING CHATBOT")
print("================================")

print("Hello Customer!")
print("How can I help you today?")

print("1. View Food Menu")
print("2. Place an Order")
print("3. Exit")
```

Sample Output:

```
================================
 WELCOME TO FOOD ORDERING CHATBOT
================================
Hello Customer!
How can I help you today?
1. View Food Menu
2. Place an Order
3. Exit
```

Explanation:

1. The print() function displays text on the screen.

2. The first three statements display the project heading.

3. The chatbot then greets the customer.

4. The final statements show the options available to the customer.

PROGRAM 2: DISPLAY FOOD MENU

Aim: To store food names and prices in a dictionary and display the complete menu.

Program:

```
menu = {
    "Pizza": 150,
    "Burger": 80,
    "Dosa": 50,
    "Sandwich": 60,
    "Biriyani": 120,
    "Fried Rice": 100
}

print("-------- FOOD MENU --------")

for food, price in menu.items():
    print(food, ":", "Rs.", price)

print("---------------------------")
```

Sample Output:

```
-------- FOOD MENU --------
Pizza : Rs. 150
Burger : Rs. 80
Dosa : Rs. 50
Sandwich : Rs. 60
Biriyani : Rs. 120
Fried Rice : Rs. 100
---------------------------
```

Explanation:

1. A dictionary named menu stores food items as keys and prices as values.

2. The items() method returns each food name together with its price.

3. The for loop visits each dictionary entry one by one.

4. The print() function displays each item and its price.

PROGRAM 3: ACCEPT FOOD ORDER

Aim: To accept a food name and check whether it is available in the menu.

Program:

```
menu = {
    "Pizza": 150,
    "Burger": 80,
    "Dosa": 50,
    "Biriyani": 120
}

food = input("Enter your food choice: ").title()

if food in menu:
    print("Your selected food:", food)
    print("Price: Rs.", menu[food])
    print("Order accepted successfully!")
else:
    print("Sorry! Food item is not available.")
```

Sample Output:

```
Enter your food choice: Pizza
Your selected food: Pizza
Price: Rs. 150
Order accepted successfully!
```

Explanation:

1. The dictionary stores the available food items and their prices.

2. input() receives the customer's choice.

3. title() standardizes the capitalization of the entered food name.

4. The if condition checks whether the food name is a key in the dictionary.

5. If it is available, the program displays its price; otherwise, it displays an unavailable message.

PROGRAM 4: CALCULATE TOTAL FOOD BILL

Aim: To add the prices of selected food items and display the total bill.

Program:

```
pizza = 150
burger = 80
dosa = 50
biriyani = 120

total = pizza + burger + dosa + biriyani

print("-------- FOOD BILL --------")
print("Pizza       : Rs.", pizza)
print("Burger      : Rs.", burger)
print("Dosa        : Rs.", dosa)
print("Biriyani    : Rs.", biriyani)
print("---------------------------")
print("Total Bill  : Rs.", total)
```

Sample Output:

```
-------- FOOD BILL --------
Pizza       : Rs. 150
Burger      : Rs. 80
Dosa        : Rs. 50
Biriyani    : Rs. 120
--------------------------
Total Bill  : Rs. 400
```

Explanation:

1. Each variable stores the price of one food item.

2. The plus (+) operator adds the four prices.

3. The calculated amount is stored in the total variable.

4. The final print() statement displays the total bill.

PROGRAM 5: FOOD ORDERING USING CONDITIONAL STATEMENTS

Aim: To allow a customer to select a food item using a menu number.

Program:

```
print("WELCOME TO FOOD ORDERING SYSTEM")
print("1. Pizza - Rs.150")
print("2. Burger - Rs.80")
print("3. Dosa - Rs.50")
print("4. Biriyani - Rs.120")

choice = int(input("Select your food (1-4): "))

if choice == 1:
    print("You selected Pizza")
    print("Price: Rs.150")
elif choice == 2:
    print("You selected Burger")
    print("Price: Rs.80")
elif choice == 3:
    print("You selected Dosa")
    print("Price: Rs.50")
elif choice == 4:
    print("You selected Biriyani")
    print("Price: Rs.120")
else:
    print("Invalid food selection")
```

Sample Output:

```
WELCOME TO FOOD ORDERING SYSTEM
1. Pizza - Rs.150
2. Burger - Rs.80
3. Dosa - Rs.50
4. Biriyani - Rs.120
Select your food (1-4): 2
You selected Burger
Price: Rs.80
```

Explanation:

1. The menu assigns a number to each food item.

2. input() receives the customer's selection, and int() converts it to an integer.

3. The if statement checks the first choice.

4. The elif statements check the other valid choices.

5. The else block handles a number that is not in the menu.

PROGRAM 6: CHATBOT CONVERSATION

Aim: To create a simple text conversation using a loop and predefined responses.

Program:

```
print("Welcome to Food Ordering Chatbot!")

while True:
    message = input("You: ").lower()

    if message == "hello":
        print("Bot: Hello! Welcome to our restaurant.")
    elif message == "menu":
        print("Bot: Available food items:")
        print("Pizza - Rs.150")
        print("Burger - Rs.80")
        print("Dosa - Rs.50")
    elif message == "pizza":
        print("Bot: Pizza selected successfully.")
    elif message == "burger":
        print("Bot: Burger selected successfully.")
    elif message == "exit":
        print("Bot: Thank you for visiting!")
        break
    else:
        print("Bot: Sorry, I don't understand.")
```

Sample Output:

```
Welcome to Food Ordering Chatbot!
You: hello
Bot: Hello! Welcome to our restaurant.
You: menu
Bot: Available food items:
Pizza - Rs.150
Burger - Rs.80
Dosa - Rs.50
You: pizza
Bot: Pizza selected successfully.
You: exit
Bot: Thank you for visiting!
```

Explanation:

1. while True keeps the chatbot running until the exit command is received.

2. input() accepts a message from the customer.

3. lower() converts the message to lowercase so that capitalization does not affect the supported commands.

4. The if and elif statements compare the message with predefined commands.

5. break ends the loop when the customer types exit.

6. This is a rule-based chatbot: it responds only to the commands written in the program.

PROGRAM 7: FOOD ORDERING USING FUNCTIONS

Aim: To separate menu display and order processing into reusable functions.

Program:

```
def display_menu():
    print("------ FOOD MENU ------")
    print("Pizza - Rs.150")
    print("Burger - Rs.80")
    print("Dosa - Rs.50")
    print("Biriyani - Rs.120")


def order_food():
    food = input("Enter food name: ").title()
```

```
    menu = {
        "Pizza": 150,
        "Burger": 80,
        "Dosa": 50,
        "Biriyani": 120
    }

    if food in menu:
        print("Selected Food:", food)
        print("Price: Rs.", menu[food])
        print("Order placed successfully!")
    else:
        print("Food item not available.")


display_menu()
order_food()
```

Sample Output:

```
------ FOOD MENU ------
Pizza - Rs.150
Burger - Rs.80
Dosa - Rs.50
Biriyani - Rs.120
Enter food name: Dosa
Selected Food: Dosa
Price: Rs. 50
Order placed successfully!
```

Explanation:

1. The def keyword defines a function.

2. display_menu() prints the available food items.

3. order_food() accepts a food name and checks it against the menu dictionary.

4. If the item exists, the program displays its name and price.

5. The final two statements call the functions so their instructions are executed.

PROGRAM 8: FOOD ORDERING WITH QUANTITY

Aim: To calculate the cost of a selected food item based on the quantity ordered.

Program:

```
menu = {
    "Pizza": 150,
    "Burger": 80,
    "Dosa": 50,
    "Biriyani": 120
}

food = input("Enter food name: ").title()

if food in menu:
    quantity = int(input("Enter quantity: "))

    if quantity > 0:
        total = menu[food] * quantity
        print("------ ORDER DETAILS ------")
        print("Food Name:", food)
        print("Quantity:", quantity)
        print("Price: Rs.", menu[food])
        print("Total Amount: Rs.", total)
        print("Order placed successfully!")
    else:
        print("Please enter a valid quantity.")
else:
    print("Food item not available.")
```

Sample Output:

```
Enter food name: Pizza
Enter quantity: 2
------ ORDER DETAILS ------
Food Name: Pizza
Quantity: 2
Price: Rs. 150
Total Amount: Rs. 300
Order placed successfully!
```

Explanation:

1. The program first checks whether the selected food is available.

2. The customer enters the quantity as an integer.

3. The condition quantity > 0 prevents zero or negative quantities from being accepted.

4. The multiplication operator (*) calculates price multiplied by quantity.

5. The program displays the food name, quantity, unit price and total amount.

PROGRAM 9: CUSTOMER DETAILS

Aim: To accept and display basic customer information for an order.

Program:

```
print("CUSTOMER DETAILS")

name = input("Enter customer name: ")
phone = input("Enter phone number: ")
address = input("Enter address: ")

print("------ CUSTOMER INFORMATION ------")
print("Customer Name:", name)
print("Phone Number:", phone)
print("Address:", address)
print("Details received successfully!")
```

Sample Output:

```
CUSTOMER DETAILS
Enter customer name: Sahana
Enter phone number: 9876543210
Enter address: Bangalore
------ CUSTOMER INFORMATION ------
Customer Name: Sahana
Phone Number: 9876543210
Address: Bangalore
Details received successfully!
```

Explanation:

1. input() collects the customer's name, phone number and address.

2. Each answer is stored in a separate variable.

3. The program displays the collected information as a simple order detail summary.

4. This sample only displays the details; it does not permanently save them to a database.

PROGRAM 10: COMPLETE BASIC FOOD ORDERING CHATBOT

Aim: To combine customer interaction, menu display, food selection, quantity calculation and final bill generation.

Program:

```
print("================================")
print("       FOOD ORDERING CHATBOT")
print("================================")
```

```
menu = {
    "Pizza": 150,
    "Burger": 80,
    "Dosa": 50,
    "Biriyani": 120,
    "Sandwich": 60
}

total = 0
name = input("Enter your name: ")

print("\nHello", name, "! Welcome.")
print("\nAvailable Food Items:")

for food, price in menu.items():
    print(food, "- Rs.", price)

while True:
    food = input("\nEnter food name or exit: ").title()

    if food == "Exit":
        break
    elif food in menu:
        quantity = int(input("Enter quantity: "))

        if quantity > 0:
            amount = menu[food] * quantity
            total = total + amount
            print(food, "added successfully.")
            print("Amount: Rs.", amount)
        else:
            print("Please enter a valid quantity.")
    else:
        print("Sorry! Food item not available.")

print("\n========== FINAL BILL ==========")
print("Customer Name:", name)
print("Total Amount: Rs.", total)
print("================================")
print("Thank you for ordering!")
print("Visit Again!")
```

Sample Output:

```
================================
       FOOD ORDERING CHATBOT
================================
Enter your name: Sahana
Hello Sahana! Welcome.

Available Food Items:
Pizza - Rs. 150
Burger - Rs. 80
Dosa - Rs. 50
Biriyani - Rs. 120
Sandwich - Rs. 60

Enter food name or exit: Pizza
Enter quantity: 2
Pizza added successfully.
Amount: Rs. 300

Enter food name or exit: Burger
Enter quantity: 1
Burger added successfully.
Amount: Rs. 80

Enter food name or exit: Exit

========== FINAL BILL ==========
Customer Name: Sahana
Total Amount: Rs. 380
================================
Thank you for ordering!
Visit Again!
```

Explanation:

1. This is the main program because it combines the earlier concepts into one ordering flow.

2. The menu dictionary stores each food item and its unit price.

3. The total variable starts at zero and stores the running bill amount.

4. The for loop displays every available food item.

5. The while loop allows the customer to order more than one item.

6. The program checks whether the entered item is available before asking for quantity.

7. The quantity must be greater than zero. The program then multiplies the unit price by quantity.

8. Each item amount is added to total. When the customer types Exit, the loop ends and the final bill is displayed.

9. This is a basic rule-based prototype. A production application would also validate all inputs, save orders in a database and connect to secure payment and restaurant systems.

9. RESULT AND DISCUSSION

The Python prototype demonstrates the main steps involved in a simple food-ordering chatbot. It displays a menu, accepts customer selections, checks item availability, calculates the cost using quantity and prints a final bill. The sample outputs show how the program responds to supported customer inputs.

The current version uses predefined commands and menu information. It is suitable for demonstrating basic Python programming and chatbot flow. It does not include a trained AI model, live menu updates, permanent order storage, online payment or real-time delivery tracking.

10. CONCLUSION

The AI Chatbot for Food Ordering project demonstrates how Python can be used to create a simple text-based ordering assistant. The program uses dictionaries, variables, input and output statements, conditional statements, loops and functions to perform food-ordering operations.

The chatbot helps customers view food items, select meals, enter quantities and calculate the total bill. It provides an organized way to demonstrate the basic food-ordering process and reduces the need for repeated manual calculations.

The project can be enhanced in the future by adding Natural Language Processing, a graphical user interface, database connectivity, customer login, secure online payments and real-time order tracking. These improvements would make the application more useful as a complete restaurant-ordering system.